

Almostashar Web - Full Code Review Report

Repository: Mohamed-ALQarram/almostashar-web | Branch: E2E_Encryption | Review date: May 14, 2026

Review objective

This report reviews the web application beyond the encryption-related area. It covers security, functional bugs, performance, architecture, duplication, maintainability, and user-experience risks across authentication, routing, API clients, SignalR chat, file/document flows, registration, admin dashboard, admin verification, and shared UI/tooling.

Important limitation: this was a static review of repository files accessible through the GitHub connector. The project was not built, executed, or tested against live APIs in this environment. Findings are therefore code-evidence based and should be validated with local build/test runs before release.

Executive summary

Severity	Count	Main risk themes
Critical	3	token theft risk, missing/unclear E2E encryption, document misclassification
High	6	production config coupling, fake admin data, broken auth UX, unsafe upload flow
Medium	9	scalability, duplicated hooks, brittle document preview, routing inconsistencies, weak validation
Low	3	linting gaps, console noise, accessibility improvements

The highest-priority issues are: tokens persisted in localStorage, plaintext chat sending on an E2E branch, uploading the national ID as a fallback practice certificate, dummy admin dashboard data, and hardcoded production endpoints.

Recommended release gate

- Do not release this branch as E2E-encrypted until client-side encryption is implemented before SignalR/API transport and verified by automated tests.
- Move refresh-token storage away from localStorage and into HttpOnly Secure SameSite cookies, or implement a temporary mitigation plan with short-lived tokens and strict CSP.
- Fix the lawyer registration document upload flow before production use, especially the practice certificate fallback and confirmPassword leak.
- Remove or clearly disable admin dashboard mock data until real endpoints are connected.
- Add CI gates for build, lint, unit tests, E2E smoke tests, security dependency audit, and upload/auth regression tests.

Reviewed scope

The review prioritized high-risk application areas and representative files from every major feature. The following files were inspected directly or through repository search results:

package.json	eslint.config.js
vite.config.js	README.md
src/main.jsx	src/App.jsx
src/routes/AppRoutes.jsx	src/routes/ProtectedRoute.jsx

src/services/api/axios.js	src/features/auth/store/authStore.js
src/features/auth/components/LoginForm.jsx	src/features/auth/components/LawyerRegisterForm.jsx
src/features/auth/components/UploadFilesForm.jsx	src/features/auth/api/lawyerRegisterApi.js
src/features/lawyer-dashboard/signalr/chatHub.js	src/features/lawyer-dashboard/hooks/useChatSignalR.js
src/features/lawyer-dashboard/hooks/useSendMessage.js	src/features/lawyer-dashboard/hooks/useTypingIndicator.js
src/features/lawyer-dashboard/hooks/useLawyerDashboard.js	src/features/lawyer-dashboard/api/lawyerDashboardApi.js
src/features/lawyer-dashboard/api/documentUploadApi.js	src/features/lawyer-dashboard/components/Chat/ChatArea.jsx
src/features/lawyer-dashboard/components/Chat/ChatList.jsx	src/features/lawyer-dashboard/store/chatStore.js
src/features/documents/hooks/usePresignedUrl.js	src/features/admin-dashboard/api/adminDashboardApi.js
src/features/admin-dashboard/hooks/useAdminDashboard.js	src/features/admin-verification/api/adminVerificationApi.js
src/features/admin-verification/hooks/useAdminVerification.js	src/features/admin-verification/components/VerificationRequestsTable.jsx
src/features/admin-verification/components/ReviewLawyerModal.jsx	src/features/admin-verification/components/DocumentPreviewModal.jsx
src/components/ui/Input.jsx	src/components/ui/Button.jsx

Findings

Each finding includes a short explanation of why it is a problem, plus impact and recommended remediation.

SEC-01 - Access and refresh tokens are persisted in browser localStorage

Critical

Category	Security
Evidence	src/features/auth/store/authStore.js stores user, accessToken, refreshToken and isAuthenticated through zustand persist using the key almostashar-auth.
Why this is a problem	localStorage is readable by any injected script, so one XSS bug can steal both the short-lived access token and the longer-lived refresh token.
Impact	Account takeover, long-lived session replay, and exposure of lawyer/client/admin data.
Recommendation	Prefer HttpOnly Secure SameSite cookies for refresh tokens, keep access tokens in memory, rotate refresh tokens, and add CSP. If localStorage must remain temporarily, minimize token lifetime and add XSS hardening.
Files	src/features/auth/store/authStore.js

SEC-02 - The E2E encryption branch still sends chat content and document metadata as plaintext

Critical

Category	Security / Product correctness
Evidence	useSendMessage builds a payload with content, messageType, documentName and documentUrl, then invokes SignalR SendMessage. ChatArea renders message.content directly from API/cache.
Why this is a problem	A branch named E2E_Encryption should not send plaintext message content to the server unless encryption is explicitly performed before transport.
Impact	The server, logs, SignalR pipeline, and any compromised backend component can see sensitive legal chat content and uploaded document references.
Recommendation	Encrypt message content and file metadata client-side before SendMessage. Store only ciphertext, nonce, algorithm/version and key identifiers server-side. Add tests that fail if plaintext is sent.
Files	src/features/lawyer-dashboard/hooks/useSendMessage.js src/features/lawyer-dashboard/components/Chat/ChatArea.jsx

BUG-01 - Optional practice certificate upload falls back to the national ID file

Critical

Category	Bug / Data integrity
Evidence	UploadFilesForm sends PracticeCertificates: practiceFile ssnFile when the practice certificate is optional and missing.
Why this is a problem	This silently uploads the wrong sensitive document under a different document type.
Impact	Admin reviewers may see duplicated/misclassified identity documents; this can corrupt verification data and expose national ID copies unnecessarily.
Recommendation	Send PracticeCertificates only when practiceFile exists. Update the API contract to allow null/omitted optional files and validate document type on the backend.
Files	src/features/auth/components/UploadFilesForm.jsx

BUG-02 - confirmPassword is not removed before registration payload is persisted and submitted**High**

Category	Bug / Privacy
Evidence	LawyerRegisterForm comments say confirmPassword is removed, but the code uses object rest without excluding it, then UploadFilesForm spreads formData into the final register payload.
Why this is a problem	The frontend sends an unnecessary password confirmation field to the backend and may persist it in client state longer than needed.
Impact	Avoidable exposure of password-derived data, noisy API payloads, and mismatch between code comments and behavior.
Recommendation	Use <code>const { confirmPassword, ...formData } = data;</code> and ensure the store never persists password fields beyond the immediate registration step.
Files	<code>src/features/auth/components/LawyerRegisterForm.jsx</code> <code>src/features/auth/components/UploadFilesForm.jsx</code>

SEC-03 - Production API and SignalR URLs are hardcoded in source code**High**

Category	Security / Configuration
Evidence	<code>axios.js</code> and <code>chatHub.js</code> both hardcode <code>https://almostashar.runasp.net</code> instead of using Vite environment variables.
Why this is a problem	Hardcoded production endpoints make it easy to accidentally test against production and hard to deploy staging, local, or disaster-recovery targets safely.
Impact	Operational mistakes, configuration drift, broken local testing, and duplicated endpoint maintenance.
Recommendation	Move base URLs to <code>VITE_API_BASE_URL</code> and <code>VITE_SIGNALR_HUB_URL</code> , validate required env vars at startup, and centralize endpoint construction in one config module.
Files	<code>src/services/api/axios.js</code> <code>src/features/lawyer-dashboard/signalr/chatHub.js</code>

BUG-03 - Admin dashboard uses dummy data and unimplemented API functions**High**

Category	Bug / Admin functionality
Evidence	<code>adminDashboardApi</code> functions are TODO placeholders; <code>useAdminDashboard</code> returns hardcoded balances, withdrawal requests, disputes, and consultants.
Why this is a problem	A protected admin dashboard should not display fake operational or financial data.
Impact	Admins can make decisions from incorrect data, especially around balances, escrow, withdrawals, and disputes.
Recommendation	Remove dashboard routes until live endpoints exist, or wire them to real APIs with loading/error states and tests. Clearly mark non-production mock pages at build time if they must exist.
Files	<code>src/features/admin-dashboard/api/adminDashboardApi.js</code> <code>src/features/admin-dashboard/hooks/useAdminDashboard.js</code>

BUG-04 - Visible authentication actions are non-functional**High**

Category	Bug / Auth UX
Evidence	The Google login button only <code>console.logs</code> , and the forgot-password link points to #.
Why this is a problem	Users are offered actions that look real but do not perform the promised auth flow.
Impact	Broken login experience, support tickets, and possible user lockout when password reset is needed.
Recommendation	Hide disabled auth providers until implemented, route forgot-password to a real page, and add tests for primary auth paths.
Files	<code>src/features/auth/components/LoginForm.jsx</code>

SEC-04 - Chat attachments have broad client-side allowed types and no visible size validation**High**

Category	Security / File handling
Evidence	ChatArea accepts image, pdf, Office files, txt, zip and rar; documentUploadApi uploads the selected file directly to /api/documents/upload.
Why this is a problem	Allowing archives and large files without client-side validation increases malware, denial-of-service, and poor UX risk. Backend validation is still required, but frontend validation reduces accidental abuse.
Impact	Large uploads, unsupported previews, risky file types, and avoidable bandwidth/storage consumption.
Recommendation	Define an allowed MIME/extension matrix, max file size, per-role limits, and user-facing validation before upload. Backend must enforce the same policy.
Files	src/features/lawyer-dashboard/components/Chat/ChatArea.jsx src/features/lawyer-dashboard/api/documentUploadApi.js

BUG-05 - Multipart handling is inconsistent between identity upload and chat upload**High**

Category	Bug / Upload reliability
Evidence	lawyerRegisterApi manually sets Content-Type: multipart/form-data, while documentUploadApi intentionally clears Content-Type so the browser sets the multipart boundary.
Why this is a problem	Manually forcing multipart/form-data can break boundary generation in some clients and makes upload behavior inconsistent.
Impact	Intermittent upload failures for identity documents while chat uploads may work.
Recommendation	Use the documentUploadApi pattern consistently: pass FormData and let axios/browser set Content-Type and boundary automatically.
Files	src/features/auth/api/lawyerRegisterApi.js src/features/lawyer-dashboard/api/documentUploadApi.js

PERF-01 - File preview creates object URLs during render and never revokes them**Medium**

Category	Performance / Memory
Evidence	FilePreview calls URL.createObjectURL(file) in the image src expression without URL.revokeObjectURL cleanup.
Why this is a problem	Each selected image can allocate browser memory that remains until page unload.
Impact	Memory growth during repeated attachment previews, especially on mobile devices or long chat sessions.
Recommendation	Create the preview URL inside useMemo/useEffect and revoke it in cleanup when the file changes or the component unmounts.
Files	src/features/lawyer-dashboard/components/Chat/ChatArea.jsx

PERF-02 - Chat list search sends requests on every keystroke and chat messages have no pagination UI**Medium**

Category	Performance / Scalability
Evidence	ChatList passes search directly into useChats; useChatMessages always requests PageSize 20 and does not expose NextCursor or load-more behavior.
Why this is a problem	Realtime search without debounce and fixed first-page message loading do not scale well as chat volume grows.
Impact	Unnecessary network traffic, rate-limit pressure, and inability to load older messages.
Recommendation	Debounce search input, use keepPreviousData or placeholderData, and implement cursor-based infinite scrolling for messages and chat lists.
Files	src/features/lawyer-dashboard/components/Chat/ChatList.jsx src/features/lawyer-dashboard/hooks/useLawyerDashboard.js src/features/lawyer-dashboard/api/lawyerDashboardApi.js

ARCH-01 - Auth store imports lawyer-dashboard SignalR code, creating cross-feature coupling**Medium**

Category	Architecture
Evidence	authStore imports stopChatHub from ../../lawyer-dashboard/signalr/chatHub and calls it during logout.
Why this is a problem	A core auth feature should not depend directly on a lawyer-dashboard implementation detail.
Impact	Circular dependency risk, harder reuse for other roles, and harder testing of auth state in isolation.
Recommendation	Move connection lifecycle code to a shared realtime service or emit a logout event that chat features subscribe to.
Files	src/features/auth/store/authStore.js src/features/lawyer-dashboard/signalr/chatHub.js

ARCH-02 - Two presigned URL hooks implement different APIs and cache policies**Medium**

Category	Architecture / Duplication
Evidence	documents/usePresignedUrl returns the raw React Query object and caches for 15 minutes; lawyer-dashboard/usePresignedUrl returns { url, isLoading } and caches for 23 hours.
Why this is a problem	Duplicate hooks with different contracts make the same concept behave differently across features.
Impact	Inconsistent link freshness, developer confusion, and harder bug fixing when presigned URL behavior changes.
Recommendation	Create one shared usePresignedUrl hook under src/features/documents or src/hooks, standardize return shape, and derive stale time from expirationMinutes.
Files	src/features/documents/hooks/usePresignedUrl.js src/features/lawyer-dashboard/hooks/usePresignedUrl.js

SEC-05 - Presigned URLs are requested for 24 hours and cached for 23 hours by default**Medium**

Category	Security / Link handling
Evidence	lawyerDashboardApi getPresignedUrl defaults expirationMinutes to 1440; lawyer-dashboard usePresignedUrl caches resolved URLs for 23 hours.
Why this is a problem	Long-lived signed links increase exposure if copied from the browser, logs, screenshots, or shared devices.
Impact	Sensitive documents remain downloadable for a long window after link exposure.
Recommendation	Use shorter expirations for sensitive documents, scope URLs to authorization checks, and tune cache time based on the returned expiration.
Files	src/features/lawyer-dashboard/api/lawyerDashboardApi.js src/features/lawyer-dashboard/hooks/usePresignedUrl.js

BUG-06 - Admin document preview identifies PDFs by searching for .pdf? inside the signed URL**Medium**

Category	Bug / Document preview
Evidence	DocumentPreviewModal uses <code>data.url.toLowerCase().includes(.pdf?)</code> to choose iframe vs image rendering.
Why this is a problem	Presigned URLs may encode the path differently or omit the file extension before the query string.
Impact	PDF documents can be rendered as broken images, blocking admin verification.
Recommendation	Use stored file metadata, MIME type, or original file extension from the file path instead of inspecting the signed URL string.
Files	<code>src/features/admin-verification/components/DocumentPreviewModal.jsx</code>

BUG-07 - Syndicate ID validation conflicts with the placeholder format**Medium**

Category	Bug / Validation
Evidence	The schema requires <code>syndicateId</code> to be a number, but the placeholder suggests a slash/year format like <code>12345/2023</code> .
Why this is a problem	A valid-looking value shown to the user will fail validation.
Impact	Registration friction and rejected valid syndicate identifiers.
Recommendation	Treat <code>syndicateId</code> as a string and validate against the actual business format.
Files	<code>src/features/auth/components/UploadFilesForm.jsx</code>

ROUTE-01 - RootRedirect has unreachable guest logic because the root route is protected first**Medium**

Category	Routing / UX
Evidence	The <code>/</code> route wraps <code>RootRedirect</code> in <code>ProtectedRoute</code> ; <code>ProtectedRoute</code> redirects unauthenticated users to <code>/guest</code> before <code>RootRedirect</code> can render <code>GuestPage</code> .
Why this is a problem	The route design contradicts the <code>RootRedirect</code> logic and makes the unauthenticated branch dead code.
Impact	Confusing routing behavior and harder maintenance when changing landing-page flows.
Recommendation	Make <code>/</code> public and let <code>RootRedirect</code> handle auth, or remove the unauthenticated branch and route directly to <code>/guest</code> .
Files	<code>src/routes/AppRoutes.jsx</code> <code>src/routes/ProtectedRoute.jsx</code>

QUALITY-01 - package.json has no test, typecheck, format, audit, or CI-oriented scripts**Medium**

Category	Quality / CI
Evidence	Scripts are limited to <code>dev</code> , <code>build</code> , <code>lint</code> , and <code>preview</code> .
Why this is a problem	A production app handling auth, legal chats, documents, and admin flows needs automated regression checks beyond linting.
Impact	Bugs in registration, chat, routing, encryption assumptions, and admin flows can ship unnoticed.
Recommendation	Add unit tests, integration tests, E2E smoke tests, <code>npm audit</code> or <code>SCA</code> , type checking if TypeScript is introduced, and a CI workflow that runs all gates.
Files	<code>package.json</code>

QUALITY-02 - Production code contains console logging for chat connection and registration/auth actions**Low**

Category	Quality / Observability
Evidence	chatHub logs connection lifecycle events; LoginForm logs Google login clicks; UploadFilesForm logs successful registration.
Why this is a problem	Console logs are noisy in production and can expose implementation details or sensitive flow timing.
Impact	Harder debugging signal-to-noise and possible leakage of operational information in shared browsers.
Recommendation	Use a small logger abstraction gated by environment and remove placeholder console logs.
Files	src/features/lawyer-dashboard/signalr/chatHub.js src/features/auth/components/LoginForm.jsx src/features/auth/components/UploadFilesForm.jsx

A11Y-01 - Some icon-only buttons lack accessible labels**Low**

Category	Accessibility / UI
Evidence	ChatArea header call/menu buttons and attachment/send controls rely on SVGs without explicit aria-label text.
Why this is a problem	Screen readers cannot reliably announce the purpose of icon-only controls.
Impact	Poor accessibility and harder keyboard/screen-reader usage.
Recommendation	Add aria-label values, semantic button names, and keyboard-focused tests with axe or eslint-plugin-jsx-a11y.
Files	src/features/lawyer-dashboard/components/Chat/ChatArea.jsx

LINT-01 - ESLint configuration is minimal and does not enforce React accessibility, import boundaries, or no-console rules**Low**

Category	Quality / Linting
Evidence	eslint.config.js includes recommended JS, react-hooks, react-refresh, and a no-unused-vars override only.
Why this is a problem	The current lint setup will not catch many issues found in this review, including broken accessibility and architectural boundary violations.
Impact	Repeat mistakes will reappear unless the tooling blocks them automatically.
Recommendation	Add jsx-a11y, import/order and boundary rules, no-console warnings, and stricter project conventions.
Files	eslint.config.js

Architecture and duplication assessment

The README describes a feature-based architecture with feature public APIs. The codebase mostly follows this direction, but several areas weaken the boundary model:

- Auth depends directly on lawyer-dashboard SignalR implementation. This couples a global auth concern to one role-specific feature.
- Presigned URL behavior is implemented twice with different return shapes and cache lifetimes.
- Admin verification imports the documents feature hook directly instead of using a stable shared document-access API.
- API endpoint constants are duplicated across axios and SignalR instead of being centrally configured.
- Admin dashboard contains design/mock data in feature hooks, which mixes presentation prototyping with production feature logic.

Security assessment

The biggest security risks are token storage, plaintext chat transport, file/link handling, and lack of security-oriented automation. The frontend alone cannot enforce all security constraints, but it should not make risky choices by default.

Area	Risk	Action
Session storage	Access and refresh tokens persisted in localStorage.	Use HttpOnly cookies for refresh tokens and memory-only access tokens.
Chat confidentiality	Messages and document metadata are sent as plaintext.	Encrypt before SendMessage/API upload metadata and store ciphertext only.
Uploads	Broad file types and inconsistent multipart handling.	Standardize upload helper, add type/size restrictions, and mirror backend validation.
Presigned URLs	24-hour signed links cached for nearly their full lifetime.	Shorten expiry and align cache time to server-returned expiration.
Tooling	No audit/security checks in scripts.	Add SCA/audit in CI and fail builds for high severity vulnerabilities.

Suggested remediation roadmap

Phase	Scope
Day 0 - release blocker fixes	SEC-01, SEC-02, BUG-01, BUG-02, BUG-03: protect session tokens, implement/verify encryption, fix registration document flow, remove fake admin data.
Day 1 - hardening	SEC-03, SEC-04, BUG-05, SEC-05: centralize environment config, standardize upload helper, enforce upload validation, shorten signed URL lifetime.
Week 1 - reliability	PERF-01, PERF-02, BUG-06, ROUTE-01: fix object URL leak, add chat pagination/debounce, harden document preview, simplify root routing.
Week 2 - maintainability	ARCH-01, ARCH-02, QUALITY-01, LINT-01: extract shared services/hooks, add tests, add CI gates and stronger lint rules.

Concrete implementation notes

Token storage target pattern

Keep refresh tokens out of JavaScript-readable storage. Use an HttpOnly Secure SameSite cookie for refresh, rotate it on every refresh, and store the access token only in memory. Add a strict Content Security Policy and avoid inline scripts.

E2E chat target pattern

Before calling SignalR SendMessage, derive or retrieve the conversation key, encrypt content locally with an authenticated encryption scheme, and send ciphertext plus metadata needed for decryption. The backend should not

receive plaintext content. Document URLs and names may also require encryption or opaque storage keys depending on the threat model.

Upload target pattern

Use a single upload utility that accepts FormData and never manually sets multipart boundaries. Validate size, type and extension before upload, show user-friendly errors, and keep backend validation authoritative.

Final assessment

The project has a reasonable feature-based structure and uses modern React, React Query, Zustand and Vite. However, this branch should not be considered production-ready for a legal platform until the critical session, encryption, document upload, and admin data issues are fixed. The codebase will benefit most from central configuration, stricter feature boundaries, shared document utilities, stronger automation, and test coverage around auth, chat, uploads, and admin flows.